CODERHOUSE

SQL

COMISIÓN 96045

# APPLE RETAIL SALES

PROYECTO FINAL

Autor: Elvis I. Labarca R

Profesor: Anderson Michel Torres

Febrero de 2026

# INTRODUCCIÓN

El presente proyecto consiste en el diseño e implementación de una base de datos relacional para la gestión de ventas en una tienda Apple Retail. El sistema permite registrar productos, categorías, tiendas, ventas y reclamos de garantía asociados a dichas ventas.

El modelo busca estructurar la información comercial de manera organizada, asegurando integridad referencial, consistencia de datos y escalabilidad para futuros módulos como clientes, métodos de pago o inventario.

## Objetivo General

Organizar y gestionar de manera estructurada la información comercial de una tienda Apple Retail, permitiendo registrar productos, categorías, tiendas, ventas y garantías, con el fin de asegurar consistencia, trazabilidad y disponibilidad confiable de los datos.

## Objetivos Específicos

- Centralizar la información de productos, ventas y garantías en un único sistema estructurado.
- Garantizar la coherencia entre las ventas realizadas y los reclamos de garantía asociados.
- Permitir el seguimiento de ventas por tienda, producto y fecha.
- Facilitar el control de productos según su categoría y precio.
- Asegurar que la información comercial pueda ser consultada de manera clara y organizada.

## Problemática

Las operaciones comerciales de una tienda retail pueden generar grandes volúmenes de datos relacionados con productos, ventas y garantías. Si estos datos se almacenan en archivos planos o sistemas no estructurados:

- Se dificulta el control de ventas.
- No existe trazabilidad clara entre ventas y reclamos.
- Se generan inconsistencias.
- Se limita el análisis comercial.

La implementación de una base de datos relacional permite centralizar la información, reducir redundancia y mejorar la gestión operativa.

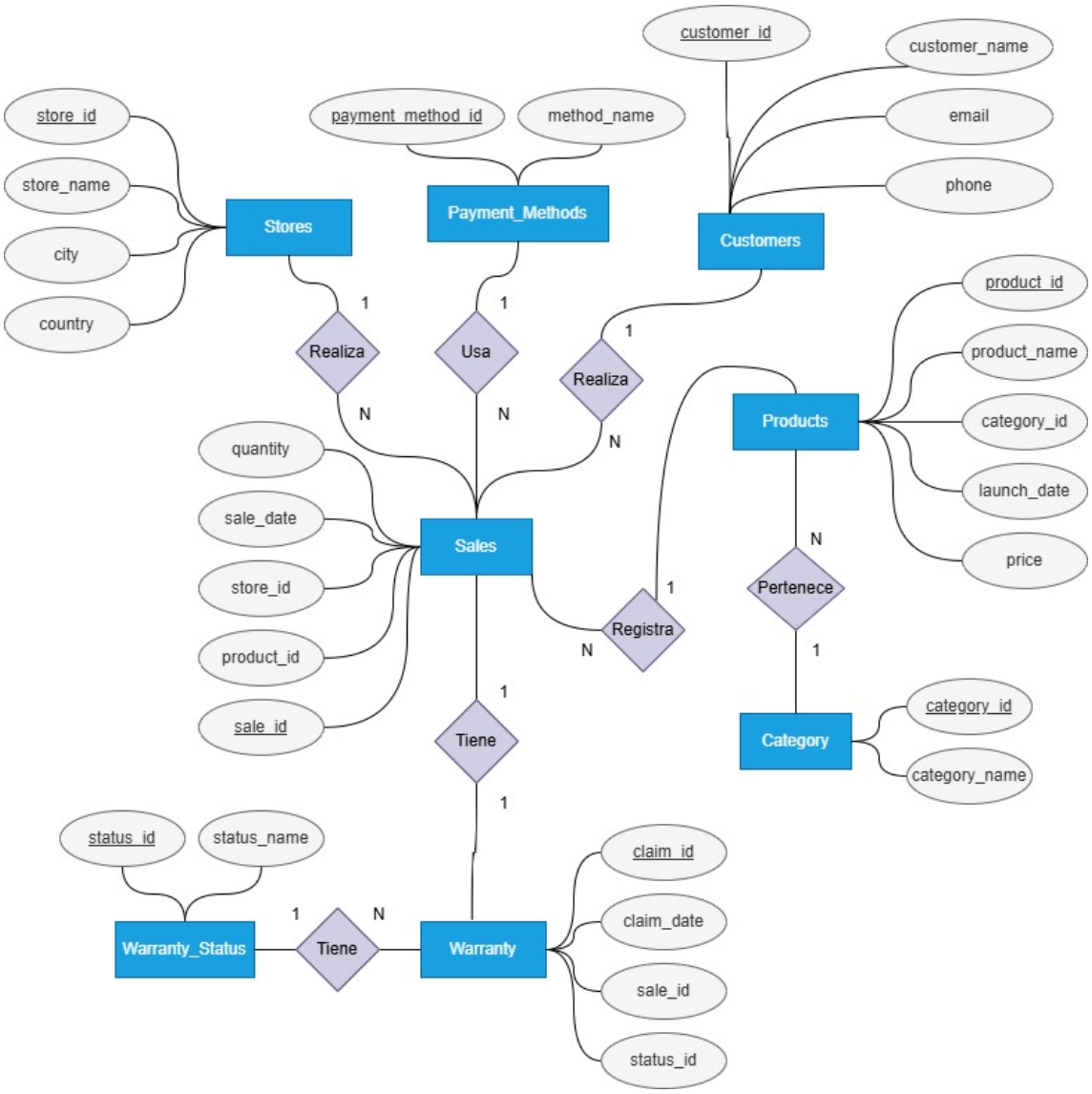
## Modelo de Negocio

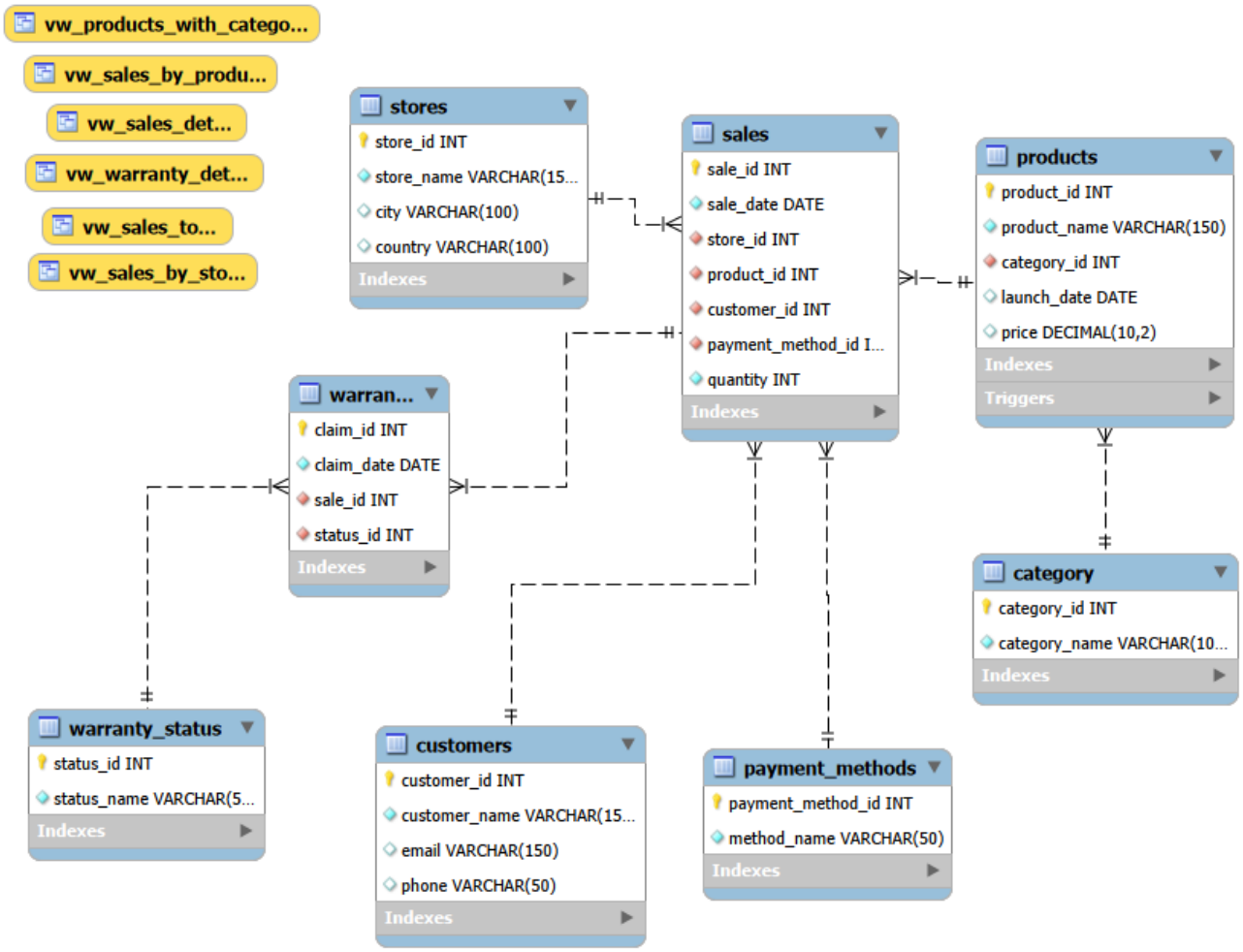
La organización modelada corresponde a una empresa retail dedicada a la venta de productos tecnológicos Apple.

El flujo básico del negocio es:

1. La empresa dispone de múltiples tiendas.
2. Cada tienda vende productos.
3. Los productos pertenecen a una categoría.
4. Cada venta registra fecha, tienda y producto vendido.
5. Algunas ventas pueden generar reclamos de garantía

## Diagramas Entidad-Relación




vw_products_with_catego...
vw_sales_by_produ...
vw_sales_det...
vw_warranty_det...
vw_sales_to...
vw_sales_by_sto...
stores
store_id INT
store_name VARCHAR(15...
city VARCHAR(100)
country VARCHAR(100)
Indexes
sales
sale_id INT
sale_date DATE
store_id INT
product_id INT
customer_id INT
payment_method_id I...
quantity INT
Indexes
products
product_id INT
product_name VARCHAR(150)
category_id INT
launch_date DATE
price DECIMAL(10,2)
Indexes
Triggers
warran...
claim_id INT
claim_date DATE
sale_id INT
status_id INT
Indexes
category
category_id INT
category_name VARCHAR(10...
Indexes
warranty_status
status_id INT
status_name VARCHAR(5...
Indexes
customers
customer_id INT
customer_name VARCHAR(15...
email VARCHAR(150)
phone VARCHAR(50)
Indexes
payment_methods
payment_method_id INT
method_name VARCHAR(50)
Indexes

**Listado de Tablas:**

**Tabla CATEGORY:**

Almacena las categorías comerciales de los productos. Permite clasificar los artículos de la empresa.

Relación: una categoría puede tener muchos productos (1:N con PRODUCTS)

| Campo | Nombre completo | Tipo | Clave |
|---|---|---|---|
| category_id | Identificador de categoría | INT | PK |
| category_name | Nombre de la categoría | VARCHAR(100) | - |

**Tabla PRODUCTS**

Almacena las categorías comerciales de los productos. Permite clasificar los artículos vendidos por la empresa.

Relación: una categoría puede tener muchos productos (1:N con PRODUCTS)

| Campo | Nombre completo | Tipo | Clave |
|---|---|---|---|
| product_id | Identificador del producto | INT | PK |
| product_name | Nombre del producto | VARCHAR(150) | - |
| category_id | Categoría del producto | INT | FK |
| launch_date | Fecha de lanzamiento | DATE | - |
| price | Precio del producto | DECIMAIL(10,2) | - |

**Tabla PAYMENT_METHODS**

| Campo | Nombre completo | Tipo | Clave |
|---|---|---|---|
| payment_method_id | Identificador del método de pago | INT | PK |
| method_name | Nombre del método de pago | VARCHAR(50) | - |

**Tabla STORES**

Registra la información de las tiendas donde se realizan las ventas, incluyendo ubicación geográfica.

Relación: una tienda puede registrar múltiples ventas (1:N con SALES).

| Campo | Nombre completo | Tipo | Clave |
|---|---|---|---|
| store_id | Identificador de tienda | INT | PK |
| store_name | Nombre de tienda | VARCHAR(150) | - |
| city | Ciudad | VARCHAR(100) | - |
| country | País | VARCHAR(100) | - |

**Tabla CUSTOMERS**

| Campo | Nombre completo | Tipo | Clave |
|---|---|---|---|
| customer_id | Identificador del cliente | INT | PK |
| customer_name | Nombre del cliente | VARCHAR(150) | - |
| email | Correo electrónico del cliente | VARCHAR(150) | - |
| phone | Teléfono del cliente | VARCHAR(50) | - |

**Tabla SALES**

Almacena las transacciones realizadas, incluyendo fecha, tienda, producto y cantidad vendida.

Relaciones: cada venta pertenece a una tienda y a un producto (N:1) y puede generar varios reclamos de garantía (1:N con WARRANTY).

| Campo | Nombre completo | Tipo | Clave |
|---|---|---|---|
| sale_id | Identificador de venta | INT | PK |
| sale_date | Fecha de venta | DATE | - |
| store_id | Tienda donde se realizó | INT | FK |
| producto_id | Producto vendido | INT | FK |
| customer_id | Cliente que realizó la compra | INT | FK |
| payment_method_id | Método de pago utilizado | INT | FK |
| quantity | Cantidad vendida | INT | - |

**Tabla WARRANTY**

Registra los reclamos de garantía asociados a una venta. Permite dar seguimiento a reparaciones o incidencias postventa.

Relación: cada reclamo está vinculado a una venta (N:1 con SALES).

| Campo | Nombre completo | Tipo | Clave |
|---|---|---|---|
| claim_id | Identificador del reclamo | INT | PK |
| claim_date | Fecha de reclamo | DATE | - |
| sale_id | Venta aociada | INT | FK |
| status_id | Estado de reparación | INT | FK |

**Tabla: WARRANTY_STATUS**

| Campo | Nombre completo | Tipo | Clave |
|---|---|---|---|
| status_id | Identificador del estado de garantía | INT | PK |
| status_name | Nombre del estado | VARCHAR(50) | - |

**Vistas:**

1. **vw_sales_detail**

**Descripción:**
Muestra el detalle completo de cada venta realizada.

**Objetivo:**
Permitir analizar ventas con toda la información relevante en una sola consulta (cliente, tienda, producto, categoría y método de pago).

**Tablas que la componen:**

- Sales
- Customers
- Stores
- Products
- Category
- Payment_Methods

2. **vw_sales_by_product**

**Descripción:**
Resume la cantidad total vendida por producto.

**Objetivo:**
Identificar los productos más vendidos.

**Tablas:**

- Sales
- Products

3. **vw_sales_by_store**

**Descripción:**
Muestra el total de ventas por tienda.

**Objetivo:**
Evaluar el rendimiento de cada tienda.

**Tablas:**

- Sales
- Stores

### 4. vw_products_with_category

**Descripción:**
Lista los productos junto con su categoría, precio y fecha de lanzamiento.

**Objetivo:**
Facilitar la consulta de productos con su clasificación.

**Tablas:**

- Products
- Category

### 5. vw_warranty_detail

**Descripción:**
Muestra el detalle de reclamos de garantía.

**Objetivo:**
Controlar el estado de las garantías y su relación con ventas y productos.

**Tablas:**

- Warranty
- Warranty_Status
- Sales
- Products
- Stores

6. **vw_sales_total**

**Descripción:**
Calcula el monto total por cada venta.

**Objetivo:**
Evitar cálculos manuales y facilitar reportes financieros.

**Tablas:**

- Sales
- Products

**Funciones:**

1. **fn_total_sale**

**Descripción:**
Calcula el total monetario de una venta específica.

**Objetivo:**
Obtener rápidamente el monto total sin repetir cálculos.

**Tablas:**

- Sales
- Products

2. **fn_customer_total_purchases**

**Descripción:**
Suma la cantidad total de productos comprados por un cliente.

**Objetivo:**
Medir el nivel de consumo de un cliente.

**Tabla:**

- Sales

3. **fn_warranty_count_by_sale**

**Descripción:**
Cuenta cuántos reclamos de garantía tiene una venta.

**Objetivo:**
Detectar productos o ventas problemáticas.

**Tabla:**

- Warranty

### 4. fn_total_sales_by_store

**Descripción:**
Calcula el total de productos vendidos por una tienda.

**Objetivo:**
Evaluar rendimiento comercial.

**Tabla:**

- Sales

**Stored Procedures:**

### 1. sp_register_sale

**Descripción:**
Registra una nueva venta.

**Objetivo:**
Centralizar el registro de ventas y evitar errores manuales.

**Tabla afectada:**

- Sales

### 2. sp_register_warranty

**Descripción:**
Registra un reclamo de garantía.

**Objetivo:**
Gestionar garantías de manera estructurada.

**Tabla:**

- Warranty

### 3. sp_update_warranty_status

**Descripción:**
Actualiza el estado de una garantía.

**Objetivo:**
Permitir el seguimiento del proceso de garantía.

**Tabla:**

- Warranty

### 4. sp_sales_by_date_range

**Descripción:**
Obtiene ventas entre dos fechas.

**Objetivo:**
Generar reportes por periodos.

**Tabla:**

- Sales

### 5. sp_sales_by_customer

**Descripción:**
Obtiene ventas de un cliente específico.

**Objetivo:**
Analizar historial de compras.

**Tabla:**

- Sales

**Triggers:**

1. **trg_validate_quantity**

**Descripción:**
Este trigger se ejecuta antes de insertar un registro en la tabla *Sales*.

**Objetivo:**
Validar que la cantidad de productos vendidos sea mayor a cero.

**Funcionamiento:**
Si el valor de quantity es menor o igual a 0, el sistema genera un error e impide la inserción del registro.

**Tabla involucrada:**

- Sales

2. **trg_validate_price**

**Descripción:**
Se ejecuta antes de insertar un producto en la tabla *Products*.

**Objetivo:**
Evitar que se registren productos con precios negativos.

**Funcionamiento:**
Si el campo price es menor a 0, el trigger bloquea la operación.

**Tabla involucrada:**

- Products

3. **trg_validate_sale_date**

**Descripción:**
Se ejecuta antes de insertar una venta.

**Objetivo:**
Asegurar que las ventas no tengan fechas futuras.

**Funcionamiento:**
Si la fecha de venta (sale_date) es mayor a la fecha actual, se genera un error.

**Tabla involucrada:**

- Sales

4. **trg_validate_warranty_date**

**Descripción:**
Se ejecuta antes de insertar un reclamo de garantía.

**Objetivo:**
Garantizar coherencia entre la fecha de venta y la fecha del reclamo.

**Funcionamiento:**
El trigger obtiene la fecha de la venta asociada y verifica que la fecha del reclamo (claim_date) no sea anterior.

**Tablas involucradas:**

- Warranty
- Sales

5. **trg_sales_audit**

**Descripción:**
Se ejecuta después de insertar una venta.

**Objetivo:**
Registrar automáticamente cada venta en una tabla de auditoría.

**Funcionamiento:**
Cada vez que se inserta una venta, se guarda un registro en la tabla *Sales_Audit* con el ID de la venta, tipo de acción y fecha.

**Tablas involucradas:**

- Sales
- Sales_Audit

**Enlace:**

https://github.com/thevisrahel/AppleRetail